



UNIVERSIDADE DE SÃO PAULO

ESCOLA POLITÉCNICA

ENGENHARIA ELÉTRICA

JEFFERSON SANTOS MONTEIRO

SOCIAL ENGINE: EVOLUÇÃO E REFATORAÇÃO DE UMA ARQUITETURA DE
REDE SOCIAL EM C++

SÃO PAULO

2026

RESUMO

Este relatório documenta o processo de desenvolvimento e a profunda refatoração arquitetural de um motor de rede social, concebido no âmbito da disciplina de Fundamentos de Software e Arquitetura de Computadores da Universidade de São Paulo. O protótipo original, desenvolvido em linguagem C++, apresentava severas limitações estruturais inerentes a exercícios acadêmicos iniciais: persistência de dados em ficheiros de texto plano, alocação de memória estática e arquitetura monolítica fortemente acoplada, resultando numa complexidade de busca linear $O(N)$ e elevado risco de fugas de memória. Com o objetivo de alinhar o projeto às exigências da Engenharia de Software moderna, o sistema foi inteiramente reestruturado. A nova arquitetura implementa um modelo Cliente-Servidor através de uma API REST headless, delega a persistência de dados para um banco de dados relacional (SQLite), garantindo integridade transacional (ACID) e eficiência de busca logarítmica $O(\log n)$, e adota o idioma RAII para a gestão automatizada e segura da memória. Os resultados evidenciam a transformação de um código fundamental num backend robusto, seguro e escalável, validando na prática o impacto vital das decisões arquiteturais e dos padrões de projeto no desempenho de sistemas computacionais complexos.

Palavras-chave: Engenharia de Software, C++, Arquitetura Cliente-Servidor, Refatoração, SQLite, Programação Orientada a Objetos.

ABSTRACT

This report documents the development process and the profound architectural refactoring of a social network engine conceived within the course Fundamentals of Software and Computer Architecture at the Universidade de São Paulo. The original prototype, developed in C++, exhibited severe structural limitations typical of early academic exercises: data persistence in plain text files, static memory allocation, and a tightly coupled monolithic architecture, resulting in linear search complexity $O(N)$ and a high risk of memory leaks. With the objective of aligning the project with the standards of modern Software Engineering, the system was completely restructured. The new architecture implements a Client-Server model through a headless REST API, delegates data persistence to a relational database (SQLite), ensuring transactional integrity (ACID) and logarithmic search efficiency $O(\log n)$, and adopts the RAII paradigm for automated and safe memory management. The results demonstrate the transformation of a foundational codebase into a robust, secure, and scalable backend, practically validating the crucial impact of architectural decisions and design patterns on the performance of complex computational systems.

Keywords: Software Engineering, C++, Client-Server Architecture, Refactoring, SQLite, Object-Oriented Programming.

LISTA DE FIGURAS E TABELAS

Figura 1: Fluxo de Execução Monolítico.....	10
Figura 2: Fluxograma de Leitura Sequencial (Legado).....	12
Figura 3: Fluxo de Decisão baseado em RTTI.....	13
Figura 4: Arquitetura da Solução.....	15
Figura 5: Ciclo de Vida de Memória (RAII vs Manual).....	17
Figura 6: Diagrama de classes da nova hierarquia.....	18
Tabela 1: Comparação entre Versão Legado e Social Engine.....	23
Figura 7: Interface da Single Page Application consumindo a API.....	25
Figura 8: Monitorização do terminal detalhando o processamento das requisições.....	26

SUMÁRIO

1. INTRODUÇÃO.....	6
1.1. Contextualização e Objetivo.....	6
1.2. Desafios para refatoração.....	7
1.3. Escopo e Requisitos do Sistema.....	8
2. ANÁLISE DA VERSÃO ANTIGA.....	10
2.1. Arquitetura Monolítica e Acoplamento de Interface.....	10
2.2. Limitações da Estrutura de Dados.....	11
2.3. Fragilidade na Persistência de Dados.....	11
2.4. Violações de POO e Polimorfismo.....	12
3. FUNDAMENTAÇÃO DA NOVA ARQUITETURA.....	14
3.1. Modelo REST API.....	14
3.2. Persistência Relacional.....	15
3.3. Gestão de Memória e RAI.....	16
4. IMPLEMENTAÇÃO DA ENGINE v1.3.5.....	18
4.1. Estrutura de Classes Refatorada.....	18
4.2. Sistema de Autenticação e Segurança.....	19
4.3. O Algoritmo de Feed.....	19
4.4. Sistema de Build e Portabilidade.....	20
4.5. Lógicas de Domínio e Inovações Arquiteturais.....	20
Governança e Controle de Acesso (RBAC).....	20
Estruturas Recursivas e Lista de Adjacência.....	21
Privacidade e Middleware Zero-Trust.....	21
Percepção de Latência e Processamento de Mídia.....	21
Internacionalização Dinâmica (i18n) e Desacoplamento de Idioma.....	21
Sincronização Assíncrona e Notificações.....	22
5. ANÁLISE COMPARATIVA.....	23
5.1. Quadro Comparativo: Legada vs. Engine.....	23
5.2. Análise de Complexidade Assintótica.....	24
6. RESULTADOS E VALIDAÇÃO.....	25
6.1. Testes de Funcionalidade.....	25
6.2. Logs de Execução e Monitorização.....	26
6.3. Análise de Estabilidade e Limitações do Frontend.....	26
6.4. Análise de Estabilidade e Limitações do Frontend.....	27
7. CONCLUSÕES.....	28
8. PRÓXIMOS PASSOS.....	29
REFERÊNCIAS BIBLIOGRÁFICAS.....	31
APÊNDICE.....	32
APÊNDICE A – Memorial Completo de Código-Fonte.....	32

1. INTRODUÇÃO

A construção de sistemas de software robustos exige uma transição clara entre os fundamentos teóricos da computação e as práticas de engenharia exigidas pelo mercado de tecnologia. Este capítulo apresenta a visão geral do projeto *Social Engine*, detalhando o cenário de sua concepção e as motivações que guiaram o seu desenvolvimento. Nas seções a seguir, são delineados os requisitos originais da aplicação, o contexto institucional em que o problema foi proposto e os objetivos que impulsionaram a evolução de um simples protótipo acadêmico para uma infraestrutura de *backend* moderna e escalável.

1.1. Contextualização e Objetivo

O presente projeto técnico nasceu de forma proativa a partir dos desafios propostos na disciplina 0323225 - Fundamentos de Software e Arquitetura de Computadores, do curso de Engenharia Elétrica da Escola Politécnica da USP, ministrada pelos professores Dr. Jaime Simão Sichman e Dr. Gustavo Pamplona Rehder.

A concepção deste trabalho está intrinsecamente alinhada às diretrizes do Projeto Piloto do Percurso de Competências da instituição. Mais do que apenas absorver teoria, este percurso pedagógico foca no desenvolvimento de habilidades práticas, exigindo do aluno a capacidade de projetar e implementar soluções computacionais. A disciplina forneceu o alicerce, abrangendo desde a complexidade de algoritmos e estruturas de dados até os paradigmas avançados de programação orientada a objetos.

Como requisito prático inicial para a consolidação destes conceitos, a disciplina propôs a criação de um sistema de rede social. O escopo original exigia a aplicação dos conceitos da POO, sendo eles encapsulamento, herança, polimorfismo e tratamento de exceções, utilizando a linguagem C++. O sistema deveria ser capaz de gerenciar perfis de utilizadores, processar conexões e garantir a persistência básica dos dados entre execuções.

O objetivo principal deste relatório, no entanto, vai além dos requisitos impostos pela atividade acadêmica. Ele visa documentar a evolução autônoma deste sistema, sendo a transição de um exercício de fixação de sintaxe e semântica de C++ para a modelagem e refatoração de um software real, respeitando rigorosos padrões de arquitetura, gestão de memória e ciclo de vida das entidades computacionais exigidos pelo mercado.

1.2. Desafios para refatoração

Embora a implementação inicial, que aqui foi denominada de “Versão Legada”, cumprisse os requisitos funcionais básicos propostos, após uma análise profunda sob a ótica da engenharia de software percebeu-se limitações estruturais consideráveis. O protótipo original operava como uma aplicação monolítica de consola, dependente do carregamento total de dados em memória RAM e utilizando um sistema de persistência sequencial baseado em ficheiros de texto.

Identificou-se que esta arquitetura, apresentava problemas significativos para uma eventual escalabilidade do sistema:

- Gargalo de Desempenho: A complexidade de busca e inserção de dados era linear $O(N)$, tornando o sistema ineficiente com o crescimento da base de utilizadores.
- Fragilidade de Dados: A persistência em texto carecia de garantias de integridade (ACID), tornando o "banco de dados" vulnerável a corrupção.
- Acoplamento: A lógica do código estava fortemente acoplada à interface de usuário e ao prompt de comandos, impossibilitando a expansão para interfaces gráficas ou web.

Motivado por estes desafios, o projeto foi expandido para além do escopo original. O que começou como um exercício de POO evoluiu para o desenvolvimento da "Social Engine", denominada pelo próprio autor, e seria uma infraestrutura de backend funcional para aplicações. Este processo envolveu uma refatoração completa, migrando de uma arquitetura monolítica para um modelo REST API (Cliente-Servidor), substituindo a gestão manual de ficheiros por um banco de dados relacional com SQLite e adotando práticas modernas de gestão de memória e compilação.

Este relatório não documenta apenas o software final, mas narra a trajetória técnica dessa evolução, comparando as decisões do projeto iniciais com a arquitetura final para evidenciar o impacto da engenharia de software na robustez, segurança e eficiência de sistemas computacionais.

1.3. Escopo e Requisitos do Sistema

O escopo do projeto *Social Engine* foi definido com o propósito de construir um motor de rede social genérico, seguro e escalável, apelidado internamente de “esqueleto”. A premissa central da refatoração foi desacoplar a infraestrutura base de regras de uma aplicação específica, permitindo que a mesma arquitetura sirva de alicerce para expansões futuras como plataformas gamificadas ou fóruns.

Para garantir a robustez exigida na projetos aplicáveis, o projeto foi guiado pelo seguinte levantamento de requisitos:

Requisitos não-funcionais para arquitetura e segurança:

- **Padronização Técnica:** O código-fonte, nomenclatura de classes e documentação técnica devem ser estritamente em inglês. O acesso a dados deve seguir o padrão DAO (*Data Access Object*), facilitando futuras migrações de banco de dados.
- **Criptografia e Sessão:** É estritamente proibido o armazenamento de senhas em texto plano. O sistema deve implementar algoritmos de *hashing* seguros aliados a um controle de sessão *stateless* via Tokens JWT.
- **Gestão de Recursos:** A aplicação deve ser imune a *memory leaks*, utilizando o paradigma RAII e dispensando a alocação estática limitante do modelo legado.

Requisitos funcionais para regras de aplicação e funcionalidades:

- **Smart Sign-in:** O sistema de *login* deve ser capaz de interpretar a entrada do utilizador através de Regex, roteando automaticamente a busca no banco de dados para E-mail, ID numérico ou *Username* alfanumérico.
- **RBAC:** As comunidades devem ser entidades independentes dos perfis de utilizadores comuns, possuindo timelines próprias e uma hierarquia rigorosa de permissões como MASTER_ADMIN, ADMIN e MEMBER, incluindo regras de sucessão de posse.
- **Privacidade:** Implementação de controle de visibilidade baseado em conexões (amigos/membros). Ao compartilhar uma publicação, o sistema deve garantir a integridade da referência: se o *post* original for deletado ou tornado privado, os compartilhamentos devem omitir os dados e exibir "Conteúdo Indisponível", evitando vazamentos de informação.

- **Feed Inteligente:** O motor deve ser capaz de compilar e ordenar cronologicamente num único fluxo as publicações provenientes tanto da rede de amigos quanto das comunidades ativas do utilizador.
- **Governança:** A estrutura de banco de dados deve prever o suporte a sistemas de denúncias para auxiliar a administração e moderação da plataforma.

2. ANÁLISE DA VERSÃO ANTIGA

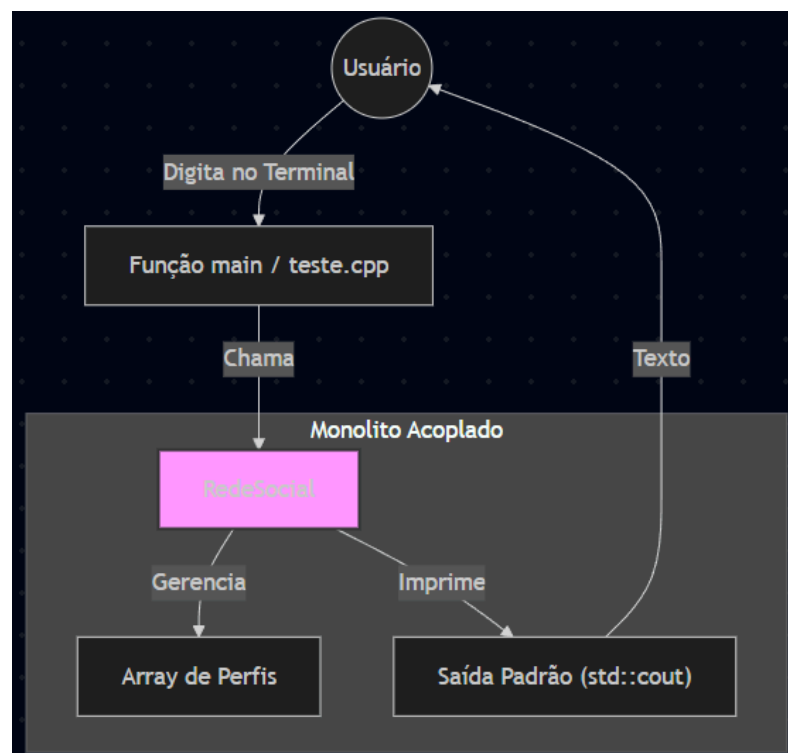
Para fundamentar as decisões arquiteturais tomadas na versão final da *Social Engine*, é necessário analisar criticamente a implementação original. O protótipo inicial, embora funcional para demonstrar a sintaxe da linguagem C++, apresentava características de design que violavam princípios modernos de engenharia de software.

2.1. Arquitetura Monolítica e Acoplamento de Interface

Na versão inicial, a classe “*RedeSocial*” acumulava responsabilidades em excesso. Além de gerenciar os dados dos perfis e as conexões, ela tratava diretamente a interação com o usuário via terminal, utilizando `cin` e `cout` da biblioteca padrão do C++.

Como mostrado na Figura 1, a lógica principal do sistema estava fortemente atrelada à forma como os dados eram exibidos. O problema dessa estrutura é a falta de isolamento entre as partes do código. Se houvesse a necessidade de migrar o projeto para uma interface gráfica ou para a web, seria necessário reescrever quase que completamente o código.

Figura 1: Fluxo de Execução Monolítico.



Fonte: Autoria própria (2026).

2.2. Limitações da Estrutura de Dados

A gestão de memória baseava-se numa alocação estática definida em tempo de compilação. A estrutura de armazenamento era um vetor de ponteiros com capacidade fixa.

Esta abordagem apresenta dois problemas críticos:

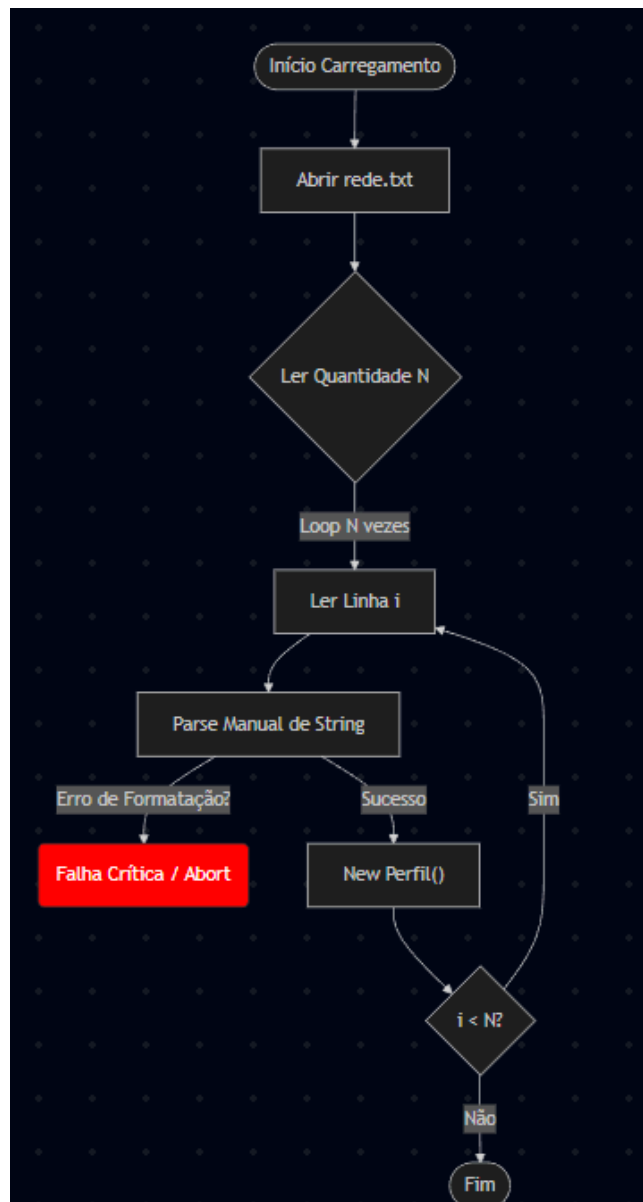
- Escalabilidade Vertical Limitada: O sistema é incapaz de crescer além do limite pré-alocado de 100 utilizadores sem recompilação.
- Risco de Stack Overflow: Ao tentar carregar todo o grafo da rede social para a memória RAM na inicialização, o sistema torna-se instável em cenários de grande volume de dados.

2.3. Fragilidade na Persistência de Dados

A persistência original utilizava serialização manual em ficheiros de texto plano. O fluxo de leitura, representado na Figura 2, era estritamente sequencial.

A fragilidade deste modelo reside na ausência de metadados e validação estrutural. A corrupção de um único byte ou uma quebra de linha acidental num registo intermediário causava a falha de leitura de todos os registos subsequentes, resultando em perda de dados. Além disso, a busca por um registo específico exigia complexidade $O(N)$ de leitura em disco.

Figura 2: Fluxograma de Leitura Sequencial (Legado)



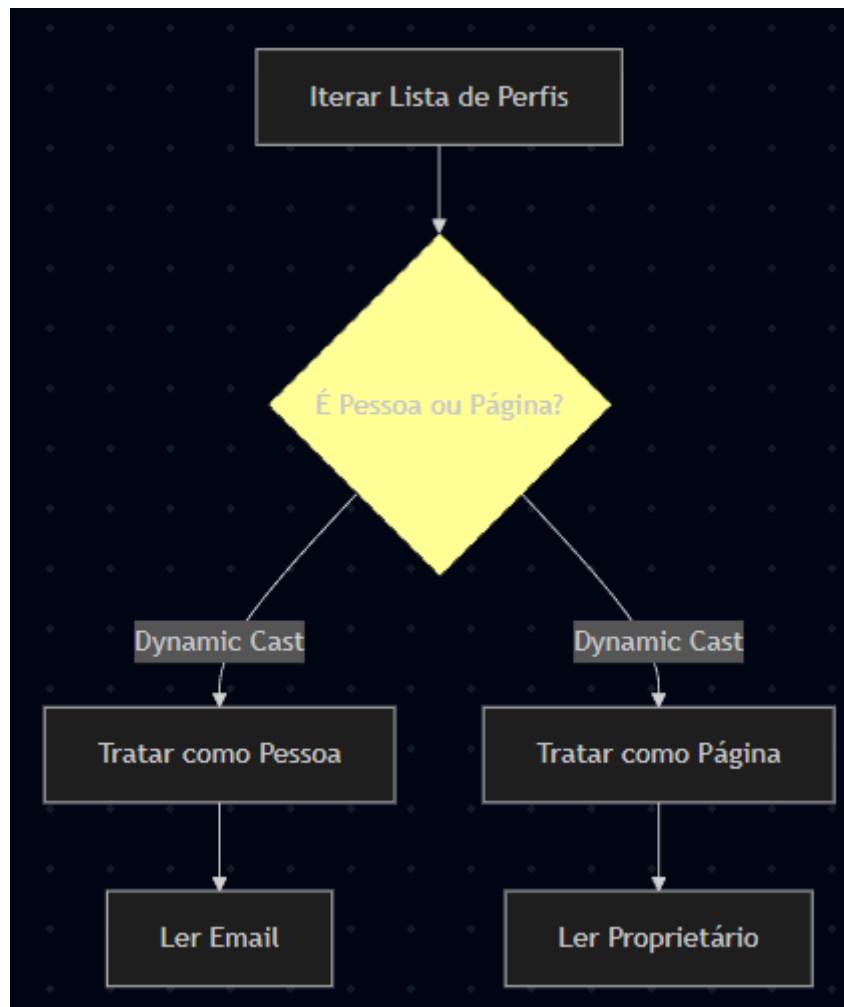
Fonte: Autoria própria (2026).

2.4. Violações de POO e Polimorfismo

A modelagem de classes original utilizava uma herança onde *Pagina* era uma subclasse de *Perfil*, mas o sistema não tirava proveito do polimorfismo real. Em vez disso, a lógica dependia de inspeção de tipos em tempo de execução, utilizando casts dinâmicos para diferenciar comportamentos.

A Figura 3 ilustra como o fluxo de controle precisava "perguntar" o tipo do objeto para decidir a ação, violando o princípio Open/Closed do SOLID. Cada novo tipo de perfil exigiria a reescrita das estruturas *condicional*.

Figura 3: Fluxo de Decisão baseado em RTTI



Fonte: Autoria própria (2026).

3. FUNDAMENTAÇÃO DA NOVA ARQUITETURA

A superação das limitações identificadas no modelo legado exigiu uma mudança de ideologia. A nova arquitetura, denominada *Social Engine*, abandona a abordagem monolítica em favor de um sistema distribuído e orientado a recursos. Este capítulo fundamenta as escolhas e os padrões de projeto adotados para garantir a performance e a integridade do sistema.

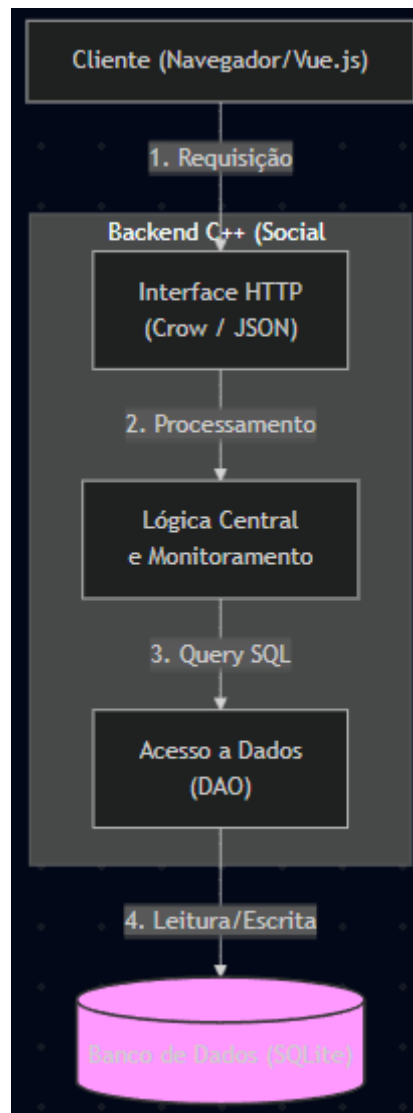
3.1. Modelo REST API

A refatoração central do projeto consistiu na adoção do modelo arquitetural REST (Representational State Transfer). Diferente da versão anterior, onde a lógica do código imprimia dados diretamente no terminal, a *Social Engine* foi desenhada como um Backend Headless.

A aplicação C++ atua agora estritamente como um servidor HTTP, utilizando o microframework Crow, o que desacopla completamente a camada de dados da camada de exibição. Nesse novo cenário, conforme ilustrado na Figura 4, o *backend* em C++ assume a responsabilidade exclusiva pelo monitoramento, englobando a lógica de feed, autenticação e grafos de amizade, e pelo acesso ao banco de dados, comunicando-se com o ambiente externo unicamente através de pacotes em formato *JSON*.

Em contrapartida, o *frontend* passa a operar como uma *Single Page Application* reativa desenvolvida em Vue.js. Seu papel é consumir a API fornecida pelo C++, transferindo para o navegador do cliente a carga de trabalho de renderizar a interface gráfica, o que liberta significativamente os recursos de processamento do servidor. Esta separação permite que a mesma *engine* sirva múltiplos clientes simultaneamente (Web, Mobile, CLI) sem necessidade de alteração no código fonte, alinhando o projeto com as práticas de mercado para sistemas distribuídos.

Figura 4: Arquitetura da Solução



Fonte: Autoria própria (2026).

3.2. Persistência Relacional

A migração de arquivos sequenciais (.txt) para o SQLite representou o maior ganho em integridade estrutural. O SQLite é uma engine de banco de dados SQL transacional que opera em estado serverless, que é ideal para a arquitetura embarcada proposta na disciplina.

A escolha por essa tecnologia justifica-se por três pilares técnicos fundamentais implementados diretamente no diretório `src/Core/Database.cpp`. O primeiro deles é a garantia de integridade referencial através do uso estrito de Foreign Keys, o que impede a geração de estados inconsistentes no sistema. Por exemplo, a tabela de publicações possui

uma restrição que vincula obrigatoriamente cada postagem ao seu respectivo autor, formalizada no esquema do banco pela diretiva `FOREIGN KEY(author_id) REFERENCES users(id)`. Essa modelagem torna matematicamente impossível a existência de uma postagem sem dono, eliminando um erro de corrupção de dados que era recorrente na persistência manual via arquivos de texto.

O segundo pilar estrutural é a otimização das buscas por meio de indexação. Enquanto a varredura em arquivos de texto plano exigia uma iteração sequencial de complexidade linear $O(N)$, a nova implementação tira proveito das estruturas B-Tree nativas do SQLite para indexar as chaves primárias. Essa mudança reduz drasticamente a quantidade de tempo necessária para a localização de perfis e postagens, alcançando um desempenho logarítmico $O(\log n)$.

Por fim, a nova arquitetura assegura a resiliência do sistema através da aplicação de transações ACID. Operações críticas, como a exclusão de uma conta, que exige a remoção em cascata do usuário, suas amizades, publicações e comentários, são encapsuladas de forma segura em blocos transacionais iniciados por `BEGIN TRANSACTION` e finalizados por `COMMIT`. Esse mecanismo garante que o comando aconteça, então ou todas as exclusões atreladas ao usuário são realizadas de uma vez, ou o estado original do banco é integralmente preservado, ignorando as mudanças. Isso previne qualquer corrupção estrutural em cenários adversos, como falhas de energia ou interrupções abruptas do processo do servidor.

3.3. Gestão de Memória e RAII

Na versão legada, a gestão de memória era manual e propensa a erros, exigindo que o desenvolvedor lembrasse de chamar `delete` para cada `new`. A nova arquitetura adota a linguagem C++ moderna RAII.

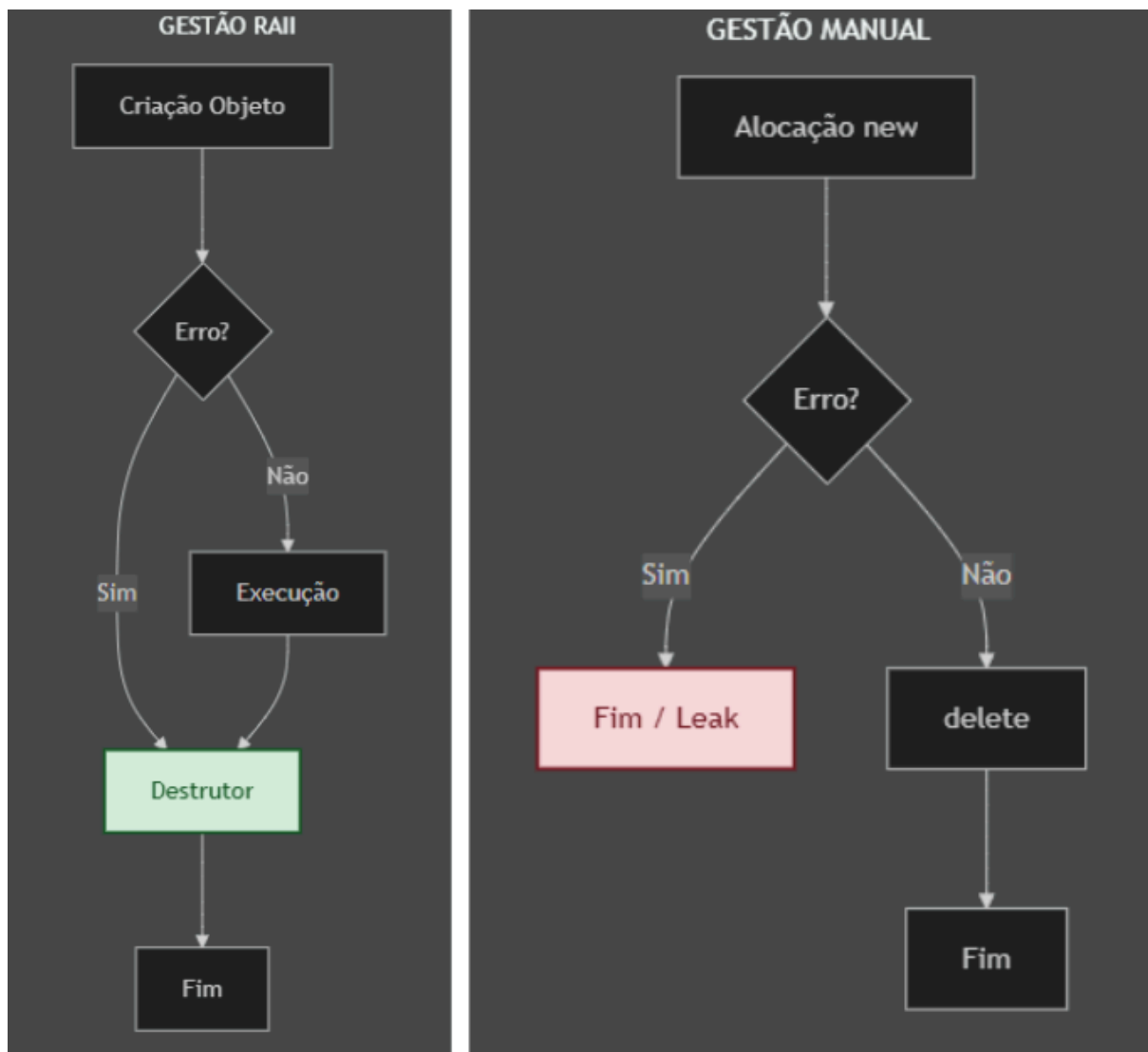
O conceito central é que a posse de um recurso, seja memória, conexão de banco ou até `handle` de arquivo, está atrelada ao tempo de vida de um objeto.

- Containers STL: Arrays, como por exemplo *Perfil***, foram substituídos por `std::vector` e `std::string`. Estes espaços gerem a sua própria alocação e desalocação de memória automaticamente. Quando um vetor sai de escopo, o seu destrutor é invocado automaticamente, liberando a memória utilizada.

- Singleton: A classe Database implementa o padrão Singleton para garantir que exista apenas uma conexão aberta com o arquivo do banco de dados durante toda a execução do programa, evitando *leaks* de descritores de arquivo.

Essa abordagem elimina a possibilidade de memory leaks na lógica, uma evolução crucial em relação aos laços manuais de destruição da versão anterior, conforme comparado na Figura 5.

Figura 5: Ciclo de Vida de Memória (RAII vs Manual)



Fonte: Autoria própria (2026).

4. IMPLEMENTAÇÃO DA ENGINE v1.3.5

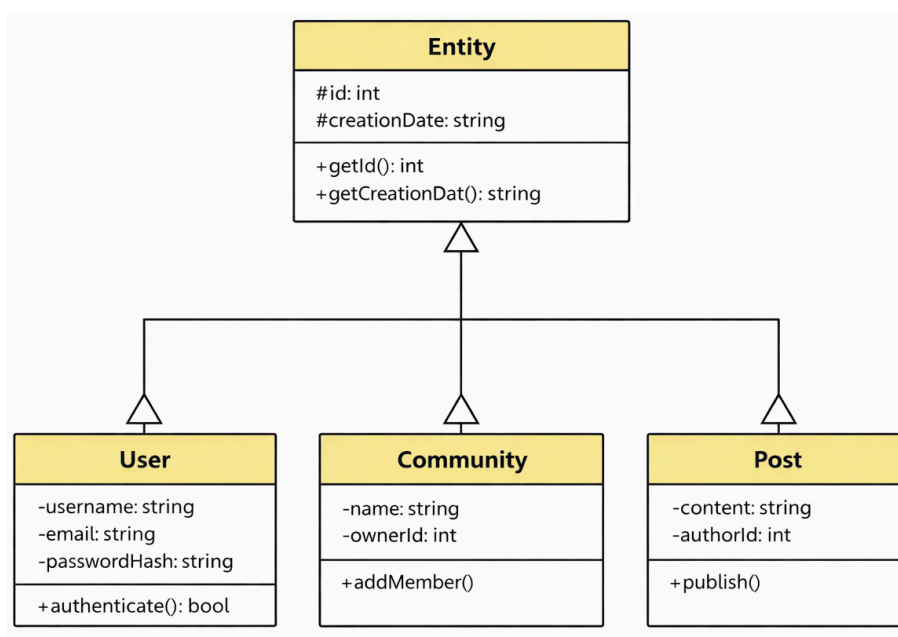
Este capítulo detalha as soluções técnicas adotadas na versão final do sistema, demonstrando a aplicação prática de conceitos de programação orientada a objetos, segurança e banco de dados.

4.1. Estrutura de Classes Refatorada

Para corrigir as violações LSP presentes no modelo legado, a arquitetura de classes foi redesenhada utilizando herança e abstração adequadas.

Foi criada uma classe abstrata chamada Entity em src/Core/Entity.h, que centraliza atributos comuns a qualquer elemento do sistema, como o ID e a data de criação. Desta classe base derivam entidades concretas como User, Community e Post, como mostrado na Figura 6 abaixo.

Figura 6: Diagrama de classes da nova hierarquia



Fonte: Autoria própria (2026).

Esta modelagem garante que uma comunidade não seja forçada a comportar-se como um perfil de usuário, possuindo agora regras próprias, como um sistema de moderação e listas de participantes.

4.2. Sistema de Autenticação e Segurança

A segurança, inexistente no protótipo original, foi tratada como requisito não-funcional. A implementação dividiu-se em duas frentes:

- Armazenamento de Senhas: Nenhuma senha é guardada em texto claro. Através da classe `Core::Crypto` utilizou-se a biblioteca OpenSSL para gerar um hash SHA-256 acoplado a um Salt criptográfico. O banco de dados armazena apenas este hash final.
- Sessão (State-less): A autenticação não guarda estado na memória do servidor. O serviço *TokenService* implementa o padrão JSON Web Token (JWT). Após o login, o cliente recebe um token assinado digitalmente, que deve ser enviado no cabeçalho HTTP em todas as requisições subsequentes para aceder a rotas protegidas.

Destaca-se ainda a implementação do Smart Sign-in no método de login, pois a API utiliza Regex para identificar automaticamente se a string enviada pelo utilizador é um e-mail, um ID numérico, ou um username, roteando a consulta SQL para a coluna correta.

4.3. O Algoritmo de Feed

O recurso mais complexo da Social Engine é a montagem do Feed de publicações. Enquanto o código antigo apenas iterava uma lista simples em memória, o novo sistema delega a complexidade matemática à engine do banco de dados relacional.

A consulta SQL elaborada realiza uma junção de múltiplos conjuntos lógicos através da cláusula UNION:

- Amigos: Posts cujos autores pertencem ao grafo de amizades do utilizador (SELECT ... FROM posts JOIN friends ...).
- Comunidades: Posts feitos dentro de comunidades nas quais o utilizador é membro.
- Públicos: Posts classificados como visíveis para toda a rede.

O resultado dessa união é ordenado de forma decrescente pela data com suporte nativo à paginação utilizando os comandos LIMIT e OFFSET, garantindo que a resposta da API seja entregue rapidamente, independente do volume total de dados armazenados.

4.4. Sistema de Build e Portabilidade

A adoção do CMake para substituir a gestão manual de compilação justificou-se pela necessidade de eliminar o acoplamento do código ao ambiente local de código e padronizar o processo de build. Essa escolha permitiu resolver problemas de compatibilidade, automatizando a configuração e a resolução de dependências do projeto em diferentes plataformas.

Através do arquivo CMakeLists.txt, o sistema passou a detectar automaticamente os recursos do sistema operacional, como o OpenSSL, e a garantir a linkagem correta da biblioteca estática do SQLite. Essa transição foi fundamental para assegurar que a aplicação pudesse ser compilada de forma idêntica e livre de erros, suportando tanto o ambiente de desenvolvimento em Windows quanto a implantação direta em servidores Linux.

4.5. Lógicas de Domínio e Inovações Arquiteturais

Além da fundação estrutural estabelecida pela adoção do modelo REST e da persistência relacional, a evolução da *Social Engine* exigiu a implementação de lógicas de domínio complexas. A arquitetura foi expandida para englobar não apenas a estabilidade do servidor, mas também a segurança de acesso, a integridade da informação e a percepção de latência pelo utilizador final. Estas inovações foram divididas em quatro pilares fundamentais:

Governança e Controle de Acesso (RBAC)

O sistema abandona a premissa de privilégios planos em favor de um modelo de Controle de Acesso Baseado em Papéis (*Role-Based Access Control* - RBAC). No escopo das comunidades, a arquitetura mapeia cada membro a um nível hierárquico específico (MASTER_ADMIN, ADMIN e MEMBER). O *backend* valida estas permissões dinamicamente, restringindo ações críticas, como a exclusão de postagens de terceiros e a transferência de posse, apenas aos utilizadores autorizados. Em paralelo, o sistema implementa o conceito de privilégio absoluto para o primeiro registo instanciado no banco de dados, referenciado como ID 1. Esta conta atua como um *Superuser* global, possuindo autorização universal para realizar auditorias, analisar denúncias no painel de *reports* e intervir na moderação de qualquer entidade da rede.

Estruturas Recursivas e Lista de Adjacência

Para suportar o engajamento através de comentários aninhados, a entidade de comentários foi modelada utilizando o padrão de banco de dados conhecido como *Adjacency List*. Cada registro possui uma chave estrangeira opcional (`parent_id`) que aponta para outro elemento da mesma tabela. Durante a requisição, a API fornece a lista plana, e a *Single Page Application* executa um algoritmo que converte estes dados numa estrutura de Grafo em tempo de execução. Esta abordagem permite a renderização gráfica recursiva de *N* níveis de profundidade sem sobrecarregar a complexidade das consultas SQL.

Privacidade e Middleware Zero-Trust

A segurança da informação não foi delegada à ocultação de elementos no *frontend*, mas sim garantida na camada de dados através de um *middleware* de validação. A *engine* aplica o princípio de *Zero-Trust* nas requisições de leitura. Se uma entidade está marcada como privada, o código em C++ intercepta o fluxo e exige a validação do grafo de amizade no exato momento da extração dos dados. Isso garante que requisições forjadas ou acessos diretos à API via terminal sejam bloqueados antes mesmo da serialização do JSON.

Percepção de Latência e Processamento de Mídia

Para atingir o padrão de reatividade das aplicações modernas, o sistema contorna a latência de rede utilizando o padrão *Optimistic UI*. Em interações de alto volume, como o envio de curtidas, a interface é atualizada instantaneamente para o utilizador, enquanto a requisição HTTP e a transação no banco de dados ocorrem de forma assíncrona em *background*. Adicionalmente, a complexidade do tráfego de arquivos binários como avatares e capas, foi mitigada através da serialização das imagens para o formato *Base64* no navegador. O servidor C++ recebe o pacote em formato de texto seguro, realiza a decodificação binária em memória, aloca o arquivo fisicamente no disco e guarda apenas o caminho de referência no SQLite, otimizando o peso do banco de dados.

Internacionalização Dinâmica (i18n) e Desacoplamento de Idioma

Uma das inovações mais notáveis da Social Engine é o suporte nativo à internacionalização com padrão i18n. Rompendo com a prática comum de injetar textos hardcoded diretamente no código, a arquitetura centraliza todas as cadeias de caracteres através do padrão de projeto Singleton na classe `Core::Translation`. O backend atua como

um provedor de dicionários, servindo as variáveis de tradução em formato JSON. No lado do cliente, a interface reativa em Vue.js consome esta estrutura e renderiza a tela instantaneamente no idioma escolhido pelo utilizador com opções de PT-BR ou EN-US, persistindo a preferência no *localStorage* do navegador.

Sincronização Assíncrona e Notificações

Para simular um ambiente de tempo real sem onerar o servidor com conexões pesadas, foi implementado no sistema uma arquitetura de *Long Polling* gerida de forma assíncrona. O módulo de notificações (*Notification.cpp*) regista eventos do sistema, como interações em publicações ou pedidos de conexão, diretamente no banco de dados. O *frontend*, operando em segundo plano, realiza consultas periódicas autônomas à API para verificar o contador de instâncias não lidas. Esta abordagem garante que o utilizador receba *feedback* visual quase instantaneamente, mantendo a carga de processamento do servidor C++ estritamente otimizada e dentro de limites previsíveis.

5. ANÁLISE COMPARATIVA

A avaliação da arquitetura de software exige métricas tangíveis. Esta secção apresenta a comparação técnica entre as decisões do protótipo e da versão final.

5.1. Quadro Comparativo: Legada vs. Engine

A tabela abaixo sintetiza os impactos arquiteturais decorrentes da refatoração abordada neste relatório.

Tabela 1: Comparação entre Versão Legada e Social Engine

Característica	Versão Legada (v0.1)	Social Engine (v1.3.5)	Impacto na Aplicação
Arquitetura	Monolito Console. A interface (cin/cout) estava fortemente acoplada à regra de negócio no ficheiro teste.cpp.	Cliente-Servidor (REST). Backend em C++ (Crow) atua como API, totalmente desacoplado da interface visual (Vue.js).	<i>Permite escalabilidade horizontal e múltiplos clientes simultâneos, simulando um ambiente de produção real.</i>
Persistência	Ficheiros de Texto Plano (.txt). Leitura e escrita sequencial manual em PersistenciaDaRede.cpp.	Banco de Dados Relacional (SQLite). Uso de transações ACID e Foreign Keys.	<i>Aumenta a integridade dos dados e elimina o risco de corrupção por formatação incorreta do texto.</i>
Alocação de Memória	Estática/Limitada. Uso de array fixo Perfil* perfis[100] e gestão manual de new/delete.	Dinâmica/RAII. Uso de std::vector, alocação sob demanda e Lazy Loading via Banco de Dados.	<i>Remove o limite arbitrário de 100 utilizadores e mitiga riscos de Memory Leaks (fugas de memória).</i>
Complexidade de Busca	Linear O(N). Para encontrar um perfil, o sistema iterava todo o array sequencialmente.	Logarítmica O(log n). Utiliza indexação B-Tree do SQL para localizar registos por ID ou Chave Única.	<i>Garante tempo de resposta constante mesmo com o crescimento da base de dados para milhões de registos.</i>
Modelagem de Classes	Herança Rígida. A classe Pagina herdava de Perfil, forçando comportamentos inadequados.	Composição e Entidades. Uso de classe base abstrata Entity, separando lógica de User e Community.	<i>Respeita o Princípio de Substituição de Liskov (LSP) e facilita a manutenção e extensão de funcionalidades.</i>
Segurança	Nula. Senhas inexistentes ou em texto claro. Validação apenas por nome.	Hash SHA-256 + Salt + JWT. Autenticação robusta e sessões geridas via Token.	<i>Requisito fundamental para segurança da informação e proteção de dados do utilizador.</i>

Fonte: Autoria própria (2026).

5.2. Análise de Complexidade Assintótica

O principal critério de complicação de performance no código base residia na complexidade de busca. Na implementação legada, para localizar o perfil de um utilizador pelo seu ID ou realizar a verificação de credenciais, o algoritmo precisava iterar sobre a totalidade do vetor carregado na memória ou ler o arquivo .txt sequencialmente do início ao fim. Consequentemente, num cenário com N utilizadores, o custo computacional de pior caso resultava numa complexidade linear $O(N)$.

Com a migração da persistência de dados para o SQLite, este cenário foi radicalmente otimizado. As colunas de identificação e as chaves primárias passaram a ser estruturadas fisicamente em disco utilizando B-Trees. Dessa forma, a complexidade matemática para encontrar um registo específico ou validar um login caiu drasticamente para $O(\log n)$. Para ilustrar o impacto prático desta mudança arquitetural, numa projeção para 1 milhão de utilizadores, a abordagem legada necessitaria de até 1.000.000 de operações de leitura para localizar um alvo no pior dos casos. Em contrapartida, a versão atual refatorada consegue resolver a mesma requisição realizando um máximo aproximado de apenas 20 operações logarítmicas.

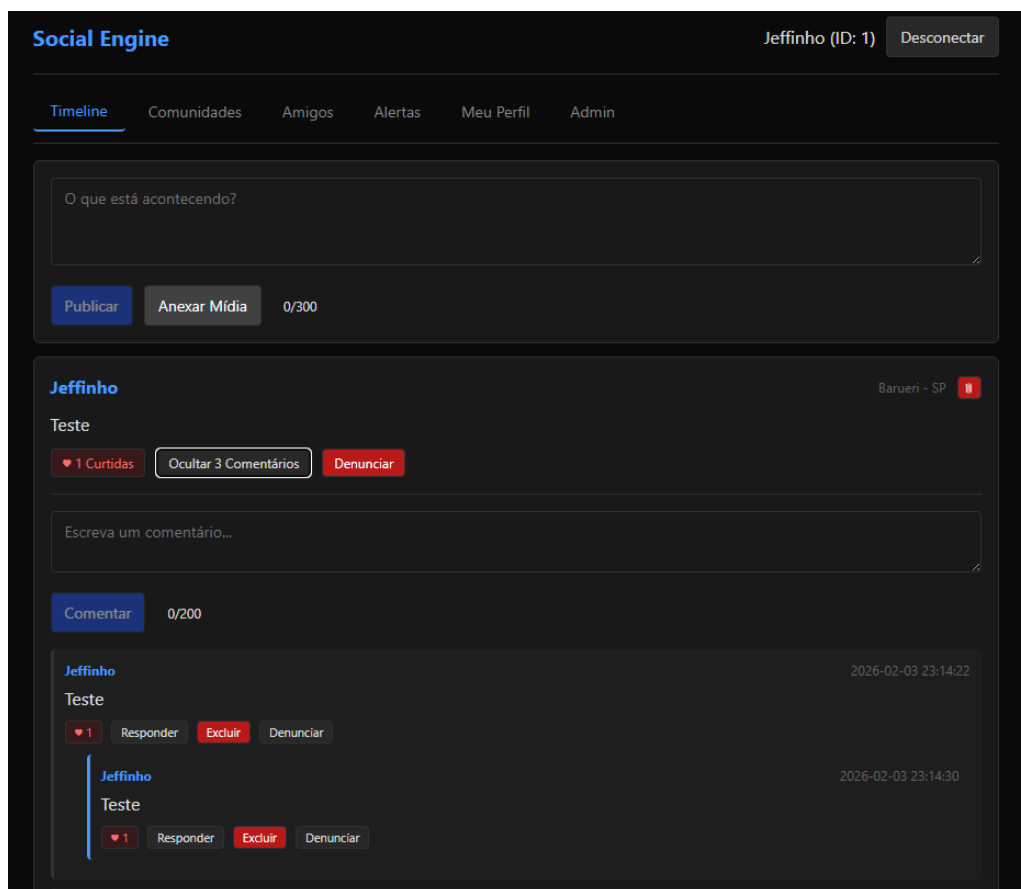
6. RESULTADOS E VALIDAÇÃO

A eficácia das decisões arquiteturais detalhadas nos capítulos anteriores só pode ser comprovada através da execução prática do software sob condições de uso reais. Este capítulo apresenta os resultados obtidos durante os testes de integração da *Social Engine*, demonstrando a viabilidade do sistema operando em um ambiente cliente-servidor e validando o correto processamento das requisições assíncronas.

6.1. Testes de Funcionalidade

O sistema foi validado executando fluxos completos de interação de forma assíncrona. Os endpoints da API responderam corretamente aos métodos HTTP GET e POST, integrando com sucesso a aplicação de frontend construída em Vue.js. Conforme ilustrado na Figura 7, as respostas em JSON comprovaram a serialização correta de objetos C++ para as estruturas dinâmicas da interface web, permitindo a renderização fluida das telas para o utilizador.

Figura 7: Interface da Single Page Application consumindo a API

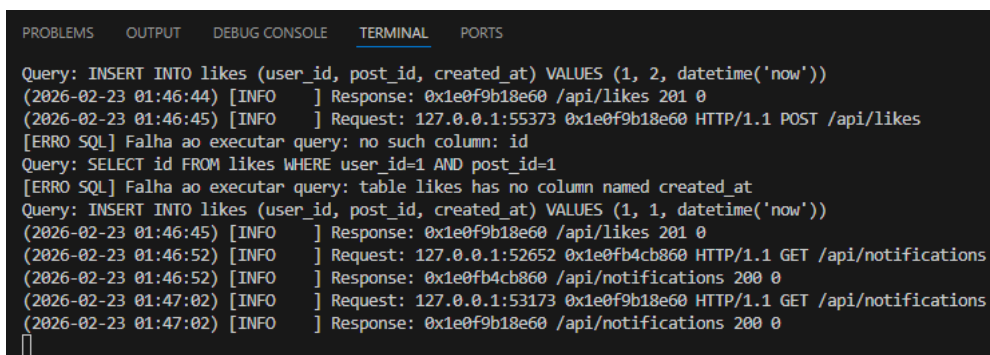


Fonte: Autoria própria (2026).

6.2. Logs de Execução e Monitorização

O microframework Crow providenciou um sistema robusto de *logging* assíncrono diretamente no terminal. O acompanhamento da execução, visível na Figura 8, permitiu rastrear os tempos de requisição (que se mantiveram na faixa de milissegundos) e monitorar o encapsulamento seguro das transações SQL a cada chamada disparada pela rede. Os registros também confirmam que a gestão de memória automatizada (RAII) procedeu corretamente com a limpeza das instâncias e liberação de recursos logo após a devolução da resposta HTTP ao cliente.

Figura 8: Monitorização do terminal detalhando o processamento das requisições



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Query: INSERT INTO likes (user_id, post_id, created_at) VALUES (1, 2, datetime('now'))
(2026-02-23 01:46:44) [INFO] Response: 0x1e0f9b18e60 /api/likes 201 0
(2026-02-23 01:46:45) [INFO] Request: 127.0.0.1:55373 0x1e0f9b18e60 HTTP/1.1 POST /api/likes
[ERRO SQL] Falha ao executar query: no such column: id
Query: SELECT id FROM likes WHERE user_id=1 AND post_id=1
[ERRO SQL] Falha ao executar query: table likes has no column named created_at
Query: INSERT INTO likes (user_id, post_id, created_at) VALUES (1, 1, datetime('now'))
(2026-02-23 01:46:45) [INFO] Response: 0x1e0f9b18e60 /api/likes 201 0
(2026-02-23 01:46:52) [INFO] Request: 127.0.0.1:52652 0x1e0fb4cb860 HTTP/1.1 GET /api/notifications
(2026-02-23 01:46:52) [INFO] Response: 0x1e0fb4cb860 /api/notifications 200 0
(2026-02-23 01:47:02) [INFO] Request: 127.0.0.1:53173 0x1e0f9b18e60 HTTP/1.1 GET /api/notifications
(2026-02-23 01:47:02) [INFO] Response: 0x1e0f9b18e60 /api/notifications 200 0
```

Fonte: Autoria própria (2026).

6.3. Análise de Estabilidade e Limitações do Frontend

Embora os testes de integração tenham comprovado a alta confiabilidade e estabilidade do *backend* em C++ no processamento das lógicas de domínio e transações com o banco de dados, a camada de apresentação apresentou oportunidades de melhoria. A interface *web* construída em Vue.js, atuando inicialmente como uma prova de conceito para o consumo da API, demonstrou instabilidades pontuais no gerenciamento de estado reativo e *bugs* de renderização durante interações muito rápidas.

Contudo, este cenário serviu para validar na prática a eficácia do isolamento proporcionado pela arquitetura Cliente-Servidor adotada. As falhas visuais ou travamentos no navegador do utilizador não propagaram erros para o servidor, não causaram fugas de memória na *engine* em C++ e não corromperam a integridade do banco de dados SQLite, provando que as camadas estão perfeitamente desacopladas.

Todo o desenvolvimento lógico da estruturação da *Social Engine* está concentrado no repositório indicado no Apêndice A.

6.4. Análise de Estabilidade e Limitações do Frontend

Para além da validação funcional descrita na seção 6.1, submeteu-se o backend a testes de carga utilizando a ferramenta Autocannon, simulando entre 50 e 100 conexões simultâneas durante intervalos de 10 segundos, com o objetivo de aferir empiricamente o comportamento do sistema sob condições de estresse e identificar gargalos não perceptíveis em testes funcionais isolados.

Em endpoints sem acesso ao banco de dados (`/health`), o servidor sustentou uma média de 9.787 requisições por segundo, com latência mediana de 9 ms, confirmando que a camada HTTP baseada no microframework Crow não representa um limitante de desempenho para o sistema.

No fluxo de autenticação (`/api/login`), que envolve consulta ao SQLite e verificação de hash de senha via OpenSSL, o throughput manteve-se em 7.695 req/s, uma redução de aproximadamente 21% em relação ao baseline, atribuída ao mutex de exclusão mútua (`std::mutex`) que serializa o acesso à instância *Singleton* do banco de dados.

Já no endpoint responsável pela composição do Feed (`/api/feed`), a análise de carga revelou uma redução mais expressiva, de aproximadamente 70% frente ao baseline (2.920 req/s). A investigação do resultado identificou duas causas concorrentes: (i) a ausência de índices explícitos nas colunas de junção das tabelas relacionais (`friendships`, `likes`, `comments`), fazendo com que as subconsultas correlacionadas presentes na query do feed — responsáveis por calcular contagem de curtidas, contagem de comentários e verificação de vínculo de amizade por publicação — exijam varredura completa de tabela; e (ii) o já mencionado mutex global de acesso ao banco.

O processo de teste de carga também expôs, de forma incidental, um defeito de produção não detectado durante os testes funcionais: uma instrução SQL malformada na função de autenticação por e-mail (`User::findByEmail`), na qual a lista de colunas da cláusula `SELECT` encontrava-se incompleta, resultando em falha de sintaxe SQL sistemática para todo login realizado por e-mail. O defeito foi identificado através da análise dos logs de erro gerados pelo próprio SQLite durante a execução do teste, e corrigido subsequentemente.

Os resultados evidenciam que, embora a arquitetura REST e o modelo de persistência relacional adotados sustentem throughput elevado em operações de leitura simples, a camada de acesso a dados apresenta oportunidades concretas de otimização —

notadamente a indexação de colunas de junção e a revisão da estratégia de concorrência no acesso ao SQLite — que são endereçadas na seção 6 (Próximos Passos).

7. CONCLUSÕES

O desenvolvimento da Social Engine representou um passo importante na transição da programação básica para as práticas reais da Engenharia de Software. A construção deste projeto deixou claro que dominar a sintaxe da linguagem C++ é apenas o começo quando se trata do desenvolvimento de sistemas completos.

A disciplina 0323225 forneceu a base teórica de Programação Orientada a Objetos, mas o trabalho prático mostrou que projetar uma arquitetura de verdade exige se preocupar com cenários de falha, gestão de memória e acesso seguro a bancos de dados. A refatoração do código legado e fortemente acoplado para uma API headless provou, na prática, como o uso correto de padrões de projeto faz toda a diferença para garantir a segurança, a escalabilidade e facilitar a manutenção do código a longo prazo.

8. PRÓXIMOS PASSOS

Tendo em vista a maturidade técnica e a estabilidade já alcançadas no backend, estão mapeadas evoluções claras para o ciclo de vida do projeto. Estas etapas foram organizadas por ordem de criticidade para a expansão contínua da plataforma, extrapolando o escopo inicial acadêmico:

- **Refatoração e Estabilização da Interface Web:** Sendo o motor em C++ considerado maduro e estável, a intervenção técnica imediata consistirá na reestruturação do cliente em Vue.js. O objetivo é sanar os bugs de renderização identificados, aprimorar o gerenciamento do estado assíncrono das requisições e garantir uma experiência de usuário fluida e condizente com o tempo de resposta logarítmico providenciado pela API.
- **Microsserviços e Inteligência Artificial:** Após a completa estabilização do frontend, planeia-se a integração da aplicação via HTTP com um serviço auxiliar escrito em Python, encarregado de executar algoritmos de *Machine Learning* para a recomendação de conteúdo e ordenação personalizada do algoritmo de feed.
- **Ecossistema Mobile Nativo:** Aproveitando o fato de a API REST já se encontrar totalmente desacoplada das telas de apresentação, o projeto expandir-se-á para a elaboração de aplicações cliente nativas ou híbridas para smartphones. Esta expansão será viabilizada sem que absolutamente nenhuma lógica do código central em C++ necessite ser alterada ou reescrita.
- **Otimização de Persistência e Concorrência:** Com base nos resultados obtidos através de testes de carga, mapeou-se a necessidade de indexação das colunas de junção relacionadas a curtidas, comentários e vínculos de amizade, bem como a avaliação de estratégias de *connection pooling* para mitigar a serialização de acesso ao banco de dados imposta pelo padrão *Singleton* atual.

REFERÊNCIAS BIBLIOGRÁFICAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Algoritmos: teoria e prática. Tradução da 2. ed. americana. Rio de Janeiro: Campus, 2002.

TANENBAUM, Andrew S. Organização estruturada de computadores. 6. ed. São Paulo: Pearson, 2013.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Prentice Hall, 2019.

STROUSTRUP, Bjarne. The C++ programming language. 4th ed. Boston: Addison-Wesley, 2013.

FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.

APÊNDICE

APÊNDICE A – Memorial Completo de Código-Fonte

Este apêndice compreende o memorial completo do código-fonte desenvolvido no projeto.

Considerando a complexidade e o volume do código, o material foi disponibilizado em repositório público de versionamento, conforme link a seguir:

Disponível em:

https://github.com/Jeff-Santz/Social_Engine-Cpp